```
    li
      - if product.available?
        = link_to product.name, product_path(product)
      - else
        = "#{product.name} out of stock"
```

The code after - is executed as Ruby, but note that no end keyword is needed—Slim inserts that for you.

Slim allows you to specify HTML classes by following a tag with a . and the class name:

```
h1.title This title has the "title" class!
```

And, in a final bit of ultracompactness, if you want to create a div with an HTML class on it, you can omit div entirely. This creates a div with the class login-form that contains two text inputs:

```
.login-form
  input type=text name=username
  input type=password name=password
```

Putting all this together, let's install Slim and reimplement the home page in app/views/store/index.html.erb using it. This will demonstrate how Rails allows us to completely replace its templating engine.

First, install slim-rails by adding it to the Gemfile:

```
gem 'slim-rails'
```

After you bundle install, your Rails app will now render files ending in slim as a Slim template. We can see this by removing app/views/store/index.html.erb and creating app/views/stores/index.slim like so:

rails7/depot_xb/app/views/store/index.slim

```
- if notice
  aside#notice = notice

h1 = t('.title_html')
```